GROUPE ISM

MINI TRAVAIL PRATIQUE

ARCHITECTURE DES APPLICATIONS

Client/serveur • 3-tiers • Couches • API

Étude de cas commune

Application Web de gestion de demandes

Module	Architecture technique des systèmes informatiques
Chapitre	Architecture des applications
Durée indicative	45 minutes à 1 heure
Travail	Individuel ou en groupe

Contexte

Une entreprise souhaite mettre en place une application Web permettant aux utilisateurs de créer un compte, se connecter, consulter leurs informations, effectuer des demandes et recevoir des notifications. L'application est accessible depuis Internet et s'appuie sur une base de données.

Réponse aux questions

1. Identifier le client, le serveur, le protocole utilisé et le rôle de chacun.

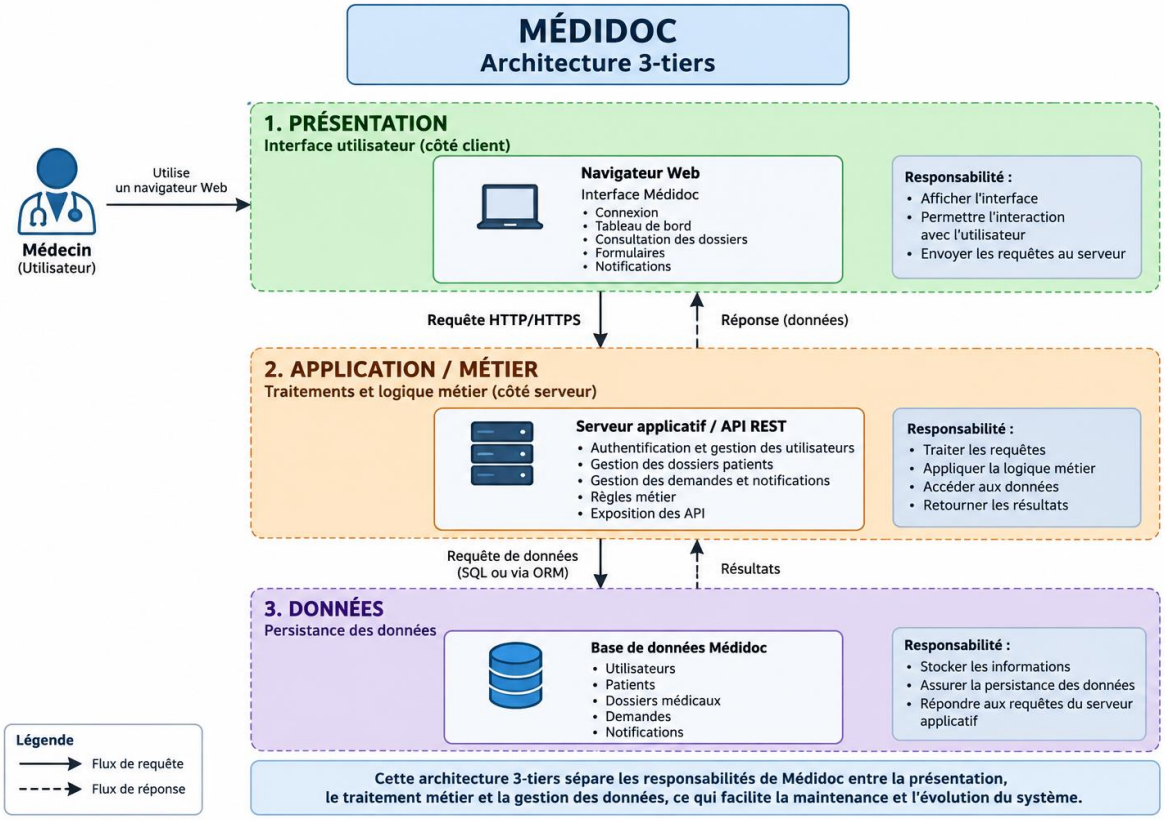
« Dans Médidoc, un utilisateur, notamment un médecin, accède à l'application Web depuis son navigateur afin de se connecter, consulter ses informations et effectuer des demandes. »le client est le navigateur Web utilisé par le médecin. Le serveur correspond à l'infrastructure applicative qui reçoit et traite les requêtes envoyées par le client. La communication entre le client et le serveur s'effectue via le protocole HTTP/HTTPS. Le client permet à l'utilisateur d'interagir avec l'application et d'afficher les résultats, tandis que le serveur assure le traitement des requêtes et l'accès aux ressources nécessaires.

Élément	Dans Médidoc	Rôle
Client	Navigateur Web du médecin	Envoie les requêtes et affiche les résultats
Serveur	Serveur de Médidoc	Reçoit les requêtes et effectue les traitements
Protocole	HTTP/HTTPS	Permet la communication entre le client et le serveur
Base de données	BD de Médidoc	Stocke les informations nécessaires à l'application

Médecin → navigateur → HTTP/HTTPS → serveur → traitement → réponse

2. Proposer une architecture 3-tiers en distinguant Présentation, Application/Métier et Données.

« Médidoc repose sur une architecture en trois tiers, organisée selon les responsabilités du système : le tier Présentation gère l'interaction avec l'utilisateur, le tier Application/Business traite les requêtes et applique la logique métier, tandis que le tier Data assure la gestion des informations persistantes. »



3. Organiser l'application en couches : présentation, logique métier et accès aux données.

Médidoc est organisé en trois couches logiques : la couche Présentation gère l'interaction avec l'utilisateur, la couche Logique métier prend en charge les traitements et l'application des règles métier, tandis que la couche Accès aux données assure les opérations de lecture et d'écriture avec la base de données. Cette séparation permet de distinguer clairement l'interface, les traitements et la gestion des données.



4. Choisir entre une architecture monolithique et une architecture microservices, puis justifier brièvement le choix.

Analyse comparative

Deux approches architecturales peuvent être envisagées pour Médidoc : **l'architecture monolithique** et **l'architecture microservices**.

L'architecture **microservices** présente l'avantage de découper l'application en services indépendants, permettant notamment une évolution et une mise à l'échelle ciblées des différentes fonctionnalités. Elle offre également une plus grande indépendance dans le déploiement des composants. Cependant, cette approche augmente la complexité technique du système : communication entre services, gestion des échanges réseau, déploiement de plusieurs composants et supervision de l'ensemble.

À l'inverse, l'architecture **monolithique** regroupe les principales fonctionnalités de l'application au sein d'un même système déployable . Elle simplifie le développement, le déploiement et l'administration de l'application, tout en permettant une organisation interne structurée en différentes couches.

Décision architecturale

Pour le périmètre actuel de Médidoc, le choix retenu est une architecture monolithique structurée en couches.

Ce choix résulte d'un **compromis entre simplicité, maintenabilité et évolutivité**. Le périmètre fonctionnel présenté dans le projet reste suffisamment centralisé pour qu'une architecture microservices n'apporte pas, à ce stade, un bénéfice proportionnel à la complexité qu'elle introduirait. L'architecture monolithique permet donc de limiter les coûts opérationnels tout en conservant une séparation claire entre les responsabilités de présentation, de logique métier et d'accès aux données.

Ce choix n'exclut pas une évolution future vers une architecture distribuée. Si Médidoc connaît une forte augmentation du nombre d'utilisateurs, une croissance importante de son périmètre fonctionnel ou un besoin de faire évoluer certaines fonctionnalités indépendamment des autres, une décomposition progressive du monolithe en microservices pourrait alors être envisagée.

Décision : Monolithe structuré en couches → simplicité et cohérence aujourd'hui, évolutivité progressive demain.

5. Proposer trois API REST cohérentes avec les fonctionnalités de l'application.

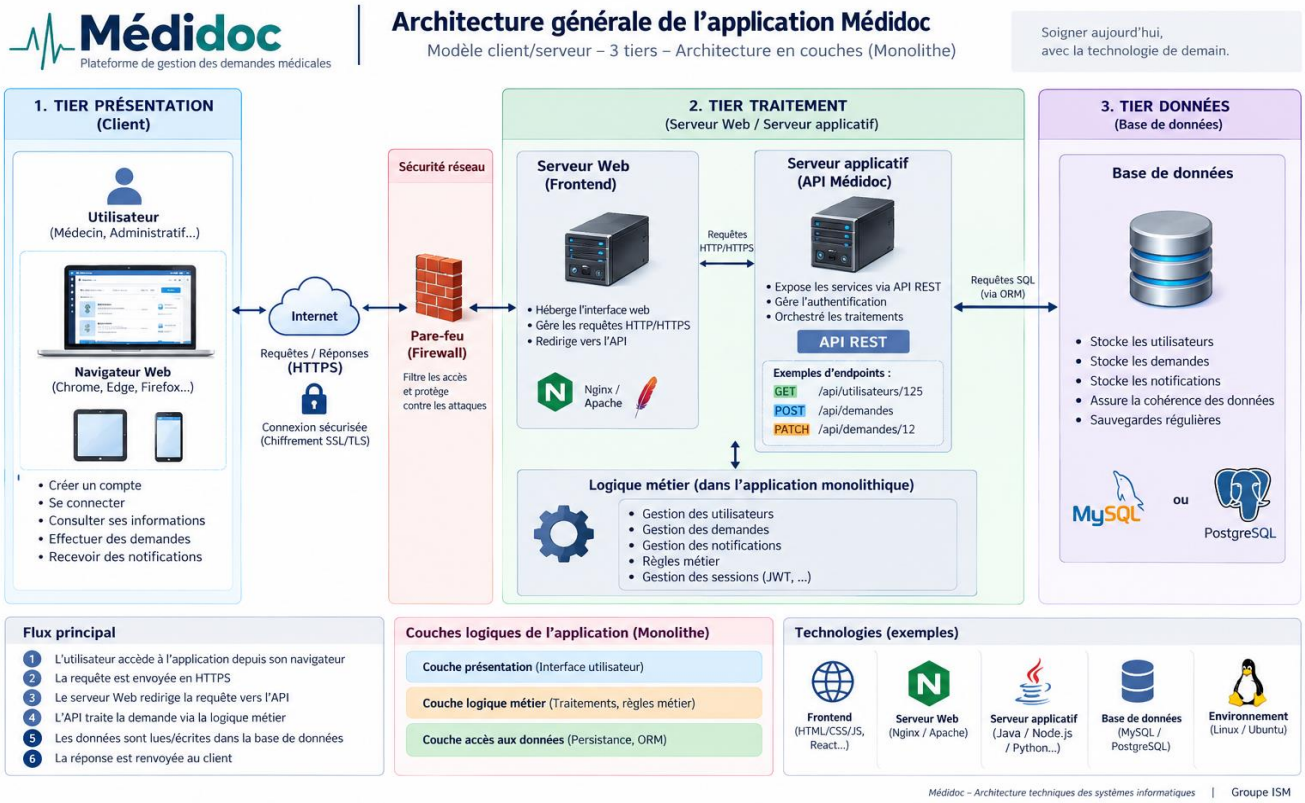
Afin d'assurer la communication entre le client Web et le serveur applicatif, Médidoc expose une API REST. Les fonctionnalités sont représentées sous forme de ressources accessibles par des endpoints et manipulées au moyen des méthodes HTTP.

Exemples d'API REST – Médidoc
Endpoints proposés et leur rôle dans l'application

Méthode HTTP	Endpoint	Ressource	Rôle / Description
GET	`/api/utilisateurs/125`	Utilisateur	Consulter les informations de l'utilisateur 125.
POST	`/api/demandes`	Demande	Créer une nouvelle demande dans le système.
PATCH	`/api/demandes/12`	Demande	Modifier certaines informations d'une demande existante (demande 12).

Ces endpoints permettent de traduire les principales opérations fonctionnelles en échanges structurés entre le client et le serveur. Ils s'intègrent au flux architectural **client → HTTP/HTTPS → API/serveur → traitement → réponse** décrit dans le sujet.

6. Réaliser un schéma simple de l'architecture générale en indiquant les principaux flux.



Synthèse architecturale et perspectives d'évolution

L'architecture proposée est adaptée au besoin de Médidoc car elle combine une architecture 3-tiers et une organisation en couches, permettant de séparer clairement la présentation, la logique métier et l'accès aux données. Le choix d'un monolithe structuré permet de répondre au périmètre actuel tout en maîtrisant la complexité technique et opérationnelle.

En cas de forte augmentation du nombre d'utilisateurs, l'architecture devrait être renforcée par des mécanismes de scalabilité tels que la mise en cache, la répartition de charge et l'optimisation ou la réplication des données. Si certains domaines fonctionnels nécessitent ensuite une évolution ou une montée en charge indépendante, une migration progressive vers des microservices pourrait être envisagée.

« On choisit l'architecture adaptée au besoin présent, mais on la conçoit avec une trajectoire d'évolution pour le besoin futur. »

Aichatou KA L3-IA